

UNIVERSIDAD SAN FRANCISCO DE QUITO
Maestría en Ciencia de Datos

TALLER 1

Implementación de un flujo ETL y Data Warehouse con datos de Stanford Puffer



Ingeniería de Datos

Estudiante: Evelyn Bermeo
Profesor: Juan Pablo Zaldumbide
Agosto de 2026

Resumen ejecutivo

El presente informe documenta la implementación de un flujo ETL desarrollado en KNIME a partir de datos de Stanford Puffer, con el objetivo de integrar cuatro fuentes heterogéneas y consolidarlas en un Data Warehouse relacional en PostgreSQL. La solución utiliza un archivo CSV, una base MySQL, una base PostgreSQL de origen y MongoDB; posteriormente, los datos son depurados, transformados, relacionados y cargados en un esquema en estrella.

Como resultado se implementó una tabla de hechos denominada fact_streaming y cinco dimensiones operativas: dim_sesion, dim_stream, dim_canal, dim_formato y dim_experimento. La carga final contiene 14.861 registros en fact_streaming. Se verificó que las cinco claves foráneas de la tabla de hechos no contienen valores nulos y que todas las referencias apuntan a registros válidos de sus respectivas dimensiones.

El flujo evidencia técnicas de ingeniería de datos como filtrado de columnas, eliminación de duplicados mediante agrupación, renombrado, manipulación de cadenas, separación de atributos, joins entre fuentes, cálculo de métricas y recuperación de claves sustitutas desde el Data Warehouse antes de cargar la tabla de hechos.

1. Objetivo y alcance

El objetivo fue construir un proceso ETL reproducible que integrara al menos cuatro fuentes de datos y culminara en un Data Warehouse con estructura dimensional. Para mantener un volumen manejable en el entorno académico, se trabajó con muestras consistentes derivadas de los archivos originales de Puffer, preservando las claves necesarias para relacionar eventos de envío, confirmación, buffer y características del video.

2. Arquitectura de la solución

La arquitectura separa claramente las fuentes operacionales del repositorio analítico. El archivo client_buffer se consume directamente desde CSV; video_sent se almacena en MySQL; video_acked se almacena en PostgreSQL; y video_size junto con ssim se consultan desde MongoDB. KNIME actúa como motor de integración y el destino final es una segunda instancia PostgreSQL denominada puffer_dw.

ARQUITECTURA GENERAL DEL PROCESO ETL

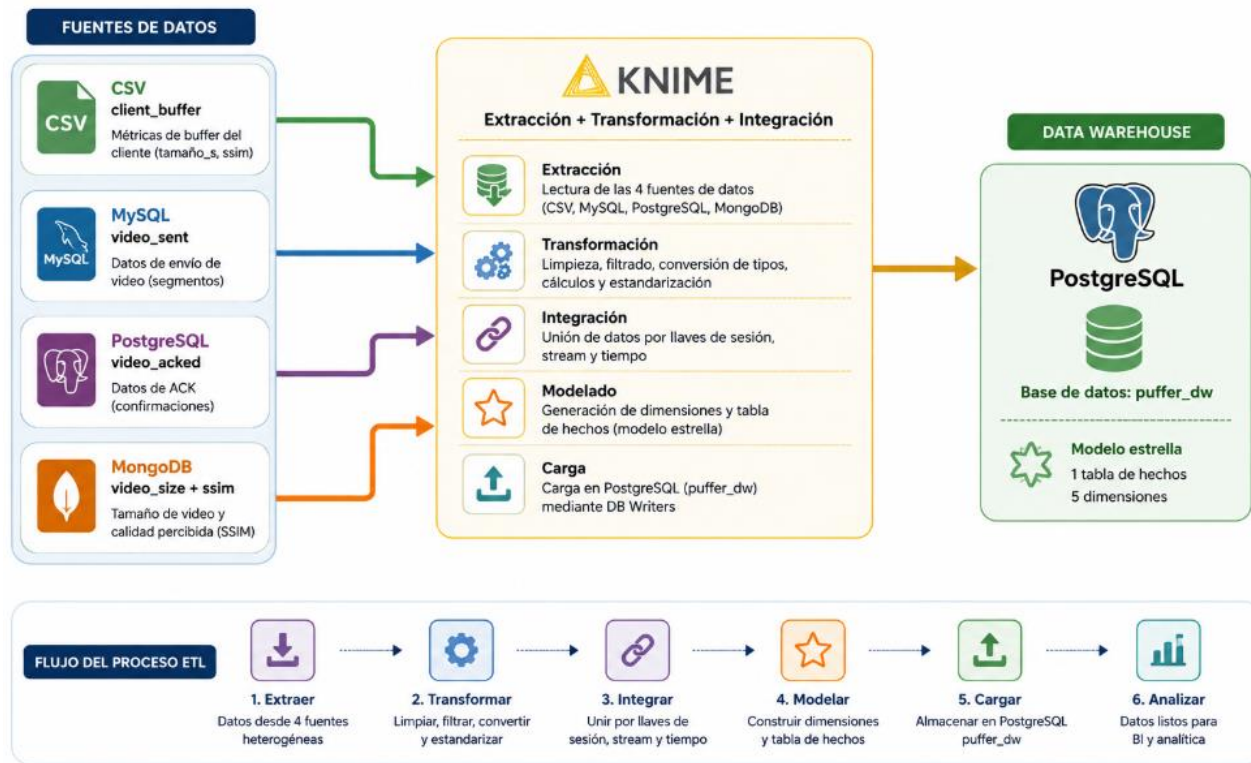


Figura 1. Arquitectura general del proceso ETL implementado.

3. Preparación del entorno y fuentes

Se utilizó Docker Compose para levantar de forma aislada los servicios de MySQL, PostgreSQL de origen, MongoDB y PostgreSQL de destino. Esta decisión permitió reproducir la infraestructura sin instalar cada gestor directamente en el sistema operativo y, al mismo tiempo, diferenciar las fuentes del Data Warehouse.

Fuente	Objeto principal	Rol
CSV	client_buffer_sample.csv	Eventos de buffer por sesión/stream
MySQL	video_sent	Chunks enviados y métricas de red
PostgreSQL origen	video_acked	Confirmaciones de chunks
MongoDB	video_size / ssim	Metadatos de formato, tamaño y SSIM
PostgreSQL DW	puffer_dw	Repositorio dimensional de destino

4. Muestreo consistente de los datos

Los archivos originales de Puffer poseen tamaños elevados, por lo que se generaron muestras consistentes. Se tomó como base una muestra de 100.000 registros de video_sent y se conservaron en los demás archivos únicamente los registros compatibles con sus claves de stream y de chunk. El resultado fue: 100.000 filas de video_sent, 99.941 de video_acked, 100.000 de client_buffer, 56.076 de video_size y 56.077 de ssim. Este enfoque evitó trabajar con subconjuntos independientes que luego no pudieran relacionarse durante los joins.

5. Proceso ETL en KNIME

El workflow de KNIME concentra la extracción, transformación, construcción de dimensiones, lectura de claves sustitutas y carga de la tabla de hechos. La Figura 2 muestra el flujo completo desarrollado.

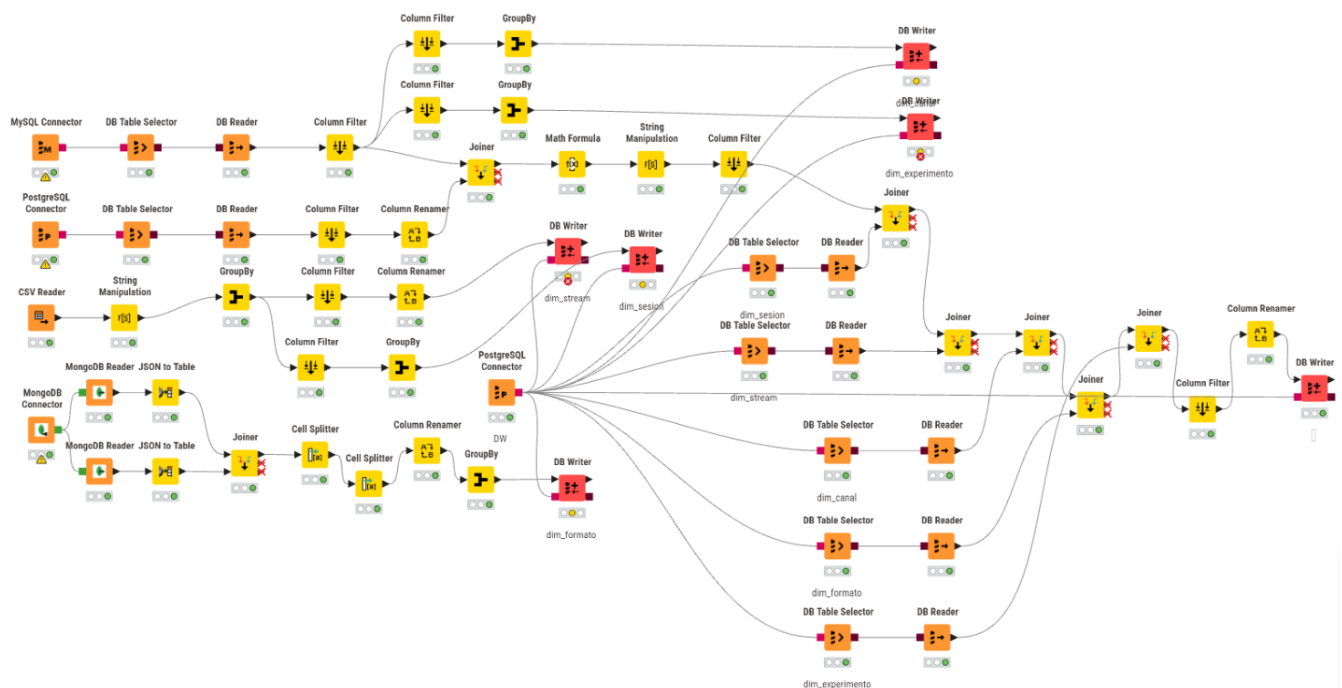


Figura 2. Workflow ETL completo en KNIME: extracción desde cuatro fuentes, transformaciones, carga de dimensiones, lookup de claves sustitutas y carga final de fact_streaming.

5.1 Extracción

La extracción se realizó con nodos específicos para cada tecnología. Para MySQL y PostgreSQL se emplearon conectores de base de datos, DB Table Selector y DB Reader. El archivo local se incorporó mediante CSV Reader. Para MongoDB se usaron MongoDB Reader y JSON to Table, transformando los documentos a una estructura tabular utilizable por KNIME.

5.2 Transformaciones relevantes

- Column Filter: selección de los atributos necesarios y eliminación de columnas auxiliares o duplicadas después de los joins.
- GroupBy: obtención de valores únicos para construir dimensiones y evitar duplicados.

- Column Renamer: homologación de nombres entre fuentes y adaptación de los campos al esquema físico del Data Warehouse.
- String Manipulation: construcción de stream_key a partir de los identificadores que representan un stream.
- Cell Splitter: descomposición del campo format para obtener width, height y crf en la dimensión de formato.
- Joiner: integración de video_sent con video_acked y posterior asociación con las dimensiones mediante claves de negocio.
- Math Formula: cálculo de ack_delay_ms a partir de la diferencia entre el instante de confirmación y el instante de envío.

5.3 Construcción y carga de dimensiones

Las dimensiones fueron construidas antes de la tabla de hechos. Cada rama del workflow selecciona la clave natural correspondiente, elimina duplicados y escribe los registros en PostgreSQL mediante DB Writer. PostgreSQL genera las claves sustitutas BIGSERIAL (id_sesion, id_stream, id_canal, id_formato e id_experimento).

Una vez cargadas las dimensiones, se volvieron a leer desde el Data Warehouse mediante DB Table Selector y DB Reader. Estas tablas se utilizaron como lookup en una secuencia de Joiner. Este paso fue esencial: al inicio, la carga de la tabla de hechos falló porque id_sesion quedaba nulo; luego se detectó que la unión debía realizarse con session_id y mantener la clave sustituta id_sesion. Después de corregir la secuencia de joins, todas las claves sustitutas quedaron presentes en las filas finales.

5.4 Preparación y carga de la tabla de hechos

Antes del DB Writer final se aplicó un Column Filter para conservar únicamente medidas y claves sustitutas. Luego se renombraron los campos para que coincidieran con el esquema físico: time a sent_time_ns, acked_time a acked_time_ns, size a size_bytes, min_rtt a min_rtt_us, rtt a rtt_us, delivery_rate a delivery_rate_bps, buffer a buffer_seconds y cum_rebuf a cum_rebuf_seconds.

Durante esta etapa se presentó un segundo error porque session_id todavía estaba incluido en el INSERT, aunque dicha columna no existe en fact_streaming. La solución fue excluir las claves naturales del flujo final y conservar solamente id_sesion, id_stream, id_canal, id_formato e id_experimento. Tras esta corrección, el DB Writer cargó correctamente 14.861 filas.

6. Modelo dimensional implementado

El modelo final corresponde a un esquema en estrella: fact_streaming se ubica en el centro y se relaciona directamente con cinco dimensiones. Las claves foráneas son obligatorias, lo que garantiza que cada hecho cargado tenga contexto de sesión, stream, canal, formato y experimento.

MODELO ESTRELLA – DATA WAREHOUSE PUFFER

Esquema implementado en PostgreSQL (puffer_dw)

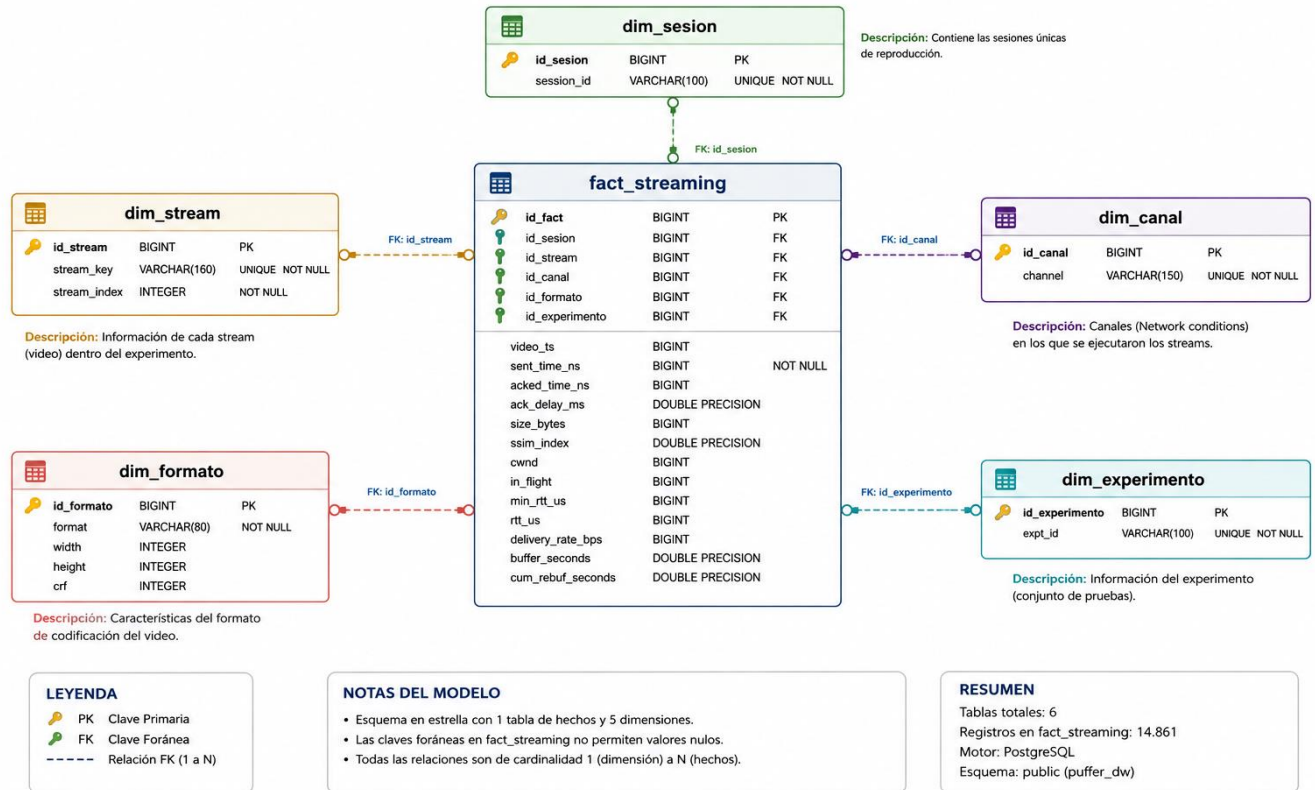


Figura 3. Modelo estrella implementado en PostgreSQL (puffer_dw), con fact_streaming como tabla central y cinco dimensiones.

6.1 Tabla de hechos

fact_streaming almacena las métricas de cada chunk de video integrado. Entre sus medidas se encuentran video_ts, sent_time_ns, acked_time_ns, ack_delay_ms, size_bytes, ssim_index, cwnd, in_flight, min_rtt_us, rtt_us, delivery_rate_bps, buffer_seconds y cum_rebuf_seconds. Además, contiene las cinco claves foráneas hacia las dimensiones.

6.2 Dimensiones

Dimensión	Clave natural / atributos	Registros cargados
dim_sesion	session_id	14
dim_stream	stream_key, stream_index	18
dim_canal	channel	2
dim_formato	format, width, height, crf	12
dim_experimento	expt_id	1

7. Validación del Data Warehouse

La validación se realizó directamente sobre PostgreSQL dentro del contenedor Docker. Primero se verificó el conteo de registros: 14 sesiones, 18 streams, 2 canales, 12 formatos, 1 experimento y 14.861 hechos.

Después se comprobó la integridad de las claves foráneas. El conteo de valores nulos en id_sesion, id_stream, id_canal, id_formato e id_experimento fue cero. Finalmente, mediante LEFT JOIN entre fact_streaming y cada dimensión se confirmó que el número de claves foráneas inválidas también fue cero en las cinco relaciones.

Validación	Resultado	Interpretación
Filas en fact_streaming	14.861	Carga final completada
FK nulas	0 en las cinco FK	No existen hechos sin dimensión
FK inválidas	0 en las cinco FK	Integridad referencial correcta

8. Resultados y conclusiones

El taller permitió implementar de extremo a extremo un flujo de ingeniería de datos con fuentes heterogéneas, transformaciones visibles en KNIME y persistencia analítica en PostgreSQL. El uso de Docker facilitó la separación de tecnologías y la reproducibilidad del entorno, mientras que KNIME permitió representar gráficamente cada etapa del ETL.

La principal dificultad técnica estuvo en la incorporación de claves sustitutas a la tabla de hechos. La solución consistió en cargar primero las dimensiones, leerlas nuevamente desde el Data Warehouse y realizar joins por claves naturales para recuperar los identificadores generados. Las validaciones finales demuestran que la solución quedó consistente: no existen claves foráneas nulas ni referencias huérfanas.

En conjunto, el resultado cumple el propósito del taller al integrar cuatro fuentes de datos y materializar un Data Warehouse dimensional listo para consultas analíticas posteriores sobre calidad de streaming, retardo, buffer, rebuffering, formato y condiciones de transmisión.

9. Anexo: resumen del flujo

Secuencia general: preparación de muestras -> carga de fuentes -> conexión desde KNIME -> extracción -> limpieza y transformación -> creación de dimensiones -> escritura en PostgreSQL DW -> lectura de dimensiones -> lookup de claves sustitutas -> selección y renombrado de medidas -> carga de fact_streaming -> validación de conteos e integridad referencial.